Pune Vegetation Monitoring & Remote-Sensing Database System

A Relational Database (MySQL 8) & Satellite Geospatial System for Longitudinal Ecological Analysis

Course Focus	Database Management Systems (DBMS) Course Project
Primary Database	MySQL 8.0 Server (Managed via MySQL Workbench 8.0)
Application Stack	Python 3.11, Streamlit, Folium, Plotly, PyMySQL, SQLAlchemy
Remote-Sensing Engine	Sentinel-2 Surface Reflectance via Google Earth Engine API
Geographic Scope	Pune Metropolitan Core & 7 Surrounding Talukas (Maharashtra, India)
Project Status	Fully Implemented, Verified & Tested (19/19 Tests Passing)

Executive Summary & Abstract:
This project presents a full-stack, relational-database-driven environmental monitoring system that bridges multispectral optical satellite remote sensing with Third Normal Form (3NF) relational database engineering. By leveraging Sentinel-2 Surface Reflectance (L2A) imagery from the European Space Agency (ESA) and Google Earth Engine (GEE), the application monitors Normalized Difference Vegetation Index (NDVI) dynamics across the climatically diverse Pune district. High-resolution raster computations remain within the satellite engine, while derived spatial bounding geometries (GeoJSON), sensor metadata, temporal observations, and summary statistics are persistently stored in MySQL 8. The system supports ACID transactional saving, 10 advanced DBMS queries (Joins, Aggregations, Grouping, Views, and Triggers), and an interactive Streamlit web dashboard.

1. Research Problem & Geographic Context

Pune district spans an ecological transition zone between the biologically diverse Western Ghats foothills (Mulshi, Maval, Bhor) and the rain-shadow semi-arid agricultural plains (Baramati, Daund, Shirur). Over recent decades, rapid urbanization, industrial zoning, infrastructure expansion, and erratic monsoon patterns have driven substantial vegetation fragmentation and microclimatic shifts. Conventional environmental studies frequently rely on ad-hoc spreadsheets or isolated raster GIS files, lacking structured relational persistence, queryability, and temporal indexing. This project resolves that limitation by establishing a unified relational database for longitudinal vegetation analysis.

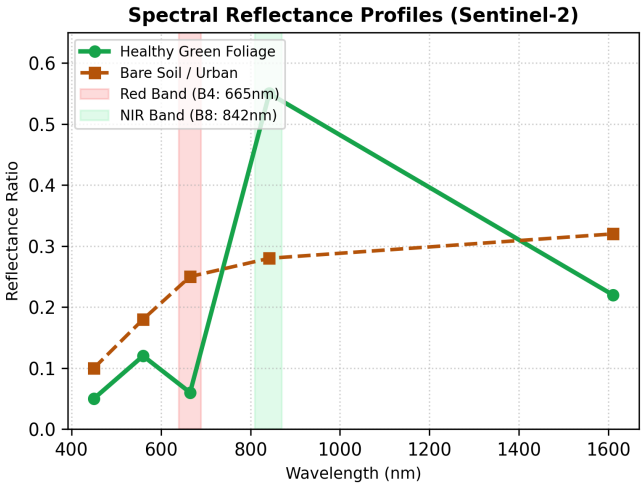
2. Satellite Remote Sensing & NDVI Mathematics

The Normalized Difference Vegetation Index (NDVI) is an internationally standardized spectral index measuring chlorophyll content and plant vigor. In healthy photosynthetic vegetation, internal leaf spongy mesophyll cell structures strongly scatter and reflect Near-Infrared (NIR) energy, while chlorophyll pigments strongly absorb Red radiation for photosynthesis.

The Mathematical Formula (Rouse et al., 1974):

NDVI = (NIR - Red) / (NIR + Red)

- Sentinel-2 Band 8 (NIR): 842 nm, 10-meter spatial resolution
- Sentinel-2 Band 4 (Red): 665 nm, 10-meter spatial resolution
- Valid Mathematical Bounds: [-1.0, +1.0]



NDVI Formula (Rouse et al., 1974)

$$NDVI = \frac{NIR\ (B8) - Red\ (B4)}{NIR\ (B8) + Red\ (B4)}$$

- < 0.10 : Bare Soil / Urban
- 0.10 - 0.25 : Built-up / Shrub
- 0.25 - 0.40 : Sparse Vegetation
- 0.40 - 0.55 : Moderate Canopy
- > 0.55 : Dense Green Foliage

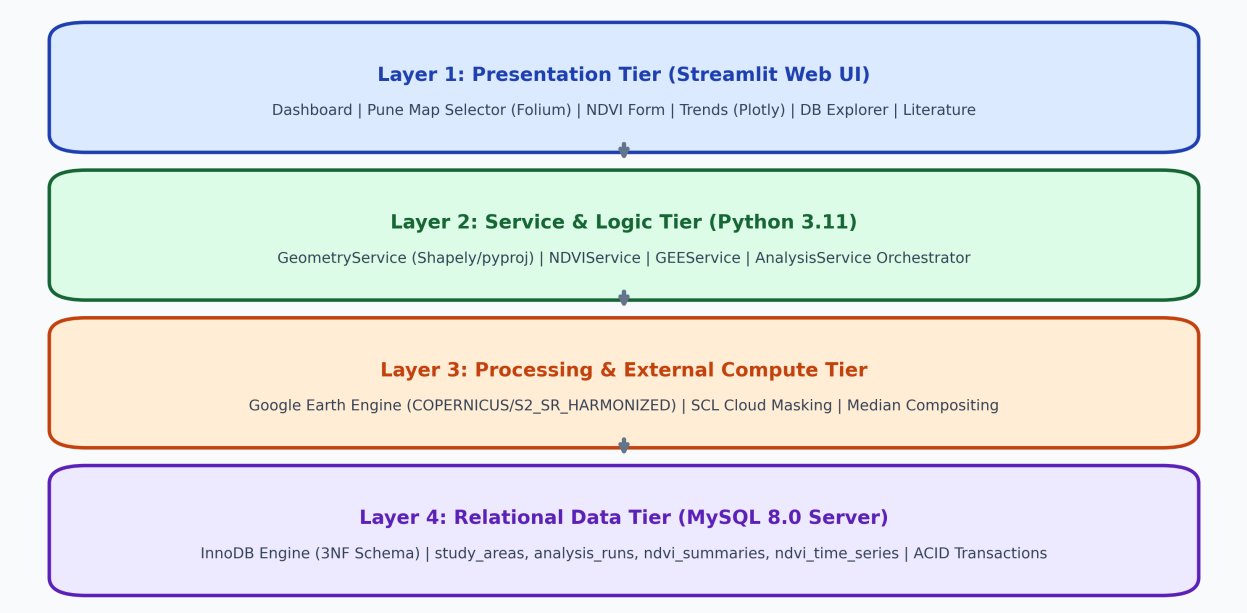
Standardized 5-Step Vegetation Classification Palette:

- **Red (< 0.10)**: Water bodies, deep shadows, or extreme non-vegetated surfaces.
- **Orange (0.10 - 0.25)**: Bare soil, rocky terrain, paved roads, and dense urban built-up areas.
- **Yellow (0.25 - 0.40)**: Sparse grass, dry shrubland, or pre-monsoon fallow agricultural fields.
- **Light Green (0.40 - 0.55)**: Moderate vegetative cover, semi-dense canopy, early crop growth.
- **Dark Green (> 0.55)**: Dense forest canopy, irrigated sugarcane/kharif crops, vigorous foliage.

Scientific Limitations: NDVI serves as an indicator of vegetative greenness, not a definitive botanical diagnostic. It does not directly measure crop yield, plant pathology/disease strains, or soil moisture without auxiliary sensors. Furthermore, heavy monsoon cloud cover during July–August requires pixel-level cloud filtering via the Scene Classification Layer (SCL).

3. System Architecture & Component Separation

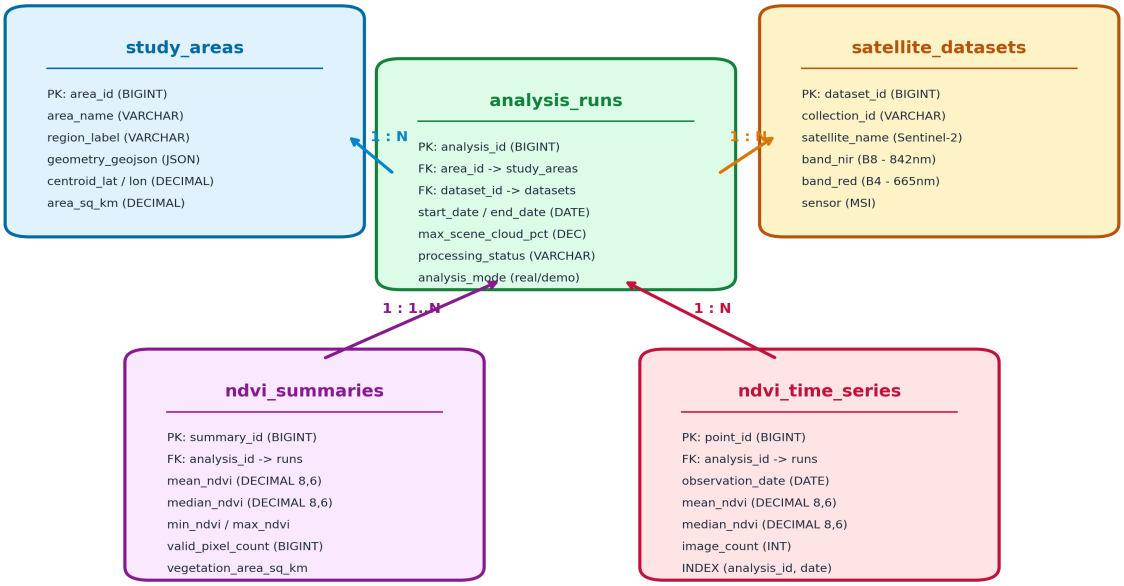
The application architecture strictly decouples satellite raster computation from the relational database. Storing multi-gigabyte raster GeoTIFF files inside relational tables leads to severe database bloat, inefficient caching, and degraded query throughput. Consequently, spatial transformations execute in Google Earth Engine, and derived summary statistics, bounding coordinates, and temporal records are preserved in MySQL 8.



Dual Processing Architecture (Real GEE vs. Demo Mode):
To guarantee operational reliability during academic presentations when cloud credentials or satellite network quotas might be unavailable, the system incorporates an integrated **Demo Mode**. When unauthenticated, the engine synthesizes biologically and climatically calibrated observations for Pune, explicitly annotating all outputs and CSV exports with DEMO DATA — NOT REAL SATELLITE RESULTS, upholding scientific integrity.

4. Relational Database Design & Schema (MySQL 8)

The database pune_vegetation_db was developed specifically for MySQL 8.0, utilizing the InnoDB storage engine for ACID compliance, foreign key integrity, and row-level concurrency.



Database Table Data Dictionary:

Table Name	Primary Key	Key Attributes	Integrity & Constraints
study_areas	area_id (BIGINT)	area_name, area_type, region_label, geometry_geojson, centroid_lat/lon, area_sq_km	CHECK(area_type IN ('preset','custom','administrative')), Spatial Indexes
satellite_datasets	dataset_id (BIGINT)	provider, collection_id (UNIQUE), satellite_name, sensor, band_nir, band_red	UNIQUE(collection_id), Defaults to Sentinel-2 Harmonized
analysis_runs	analysis_id (BIGINT)	area_id (FK), dataset_id (FK), start_date, end_date, max_scene_cloud_pct, status	FK -> study_areas, datasets; CHECK(start_date <= end_date); CHECK(cloud <= 100)
ndvi_summaries	summary_id (BIGINT)	analysis_id (FK), mean_ndvi, median_ndvi, min_ndvi, max_ndvi, valid_pixel_count	FK -> analysis_runs (ON DELETE CASCADE); CHECK(mean_ndvi BETWEEN -1 AND 1)
ndvi_time_series	point_id (BIGINT)	analysis_id (FK), observation_date, mean_ndvi, median_ndvi, image_count	INDEX(analysis_id, observation_date); UNIQUE(analysis_id, date); ON DELETE CASCADE
audit_logs	log_id (BIGINT)	action_type, table_name, record_id, details, logged_at	Populated automatically via Trigger on analysis creation

ACID Transaction Management:
Persisting an analysis run spans multiple tables: inserting the job definition in analysis_runs, persisting aggregated statistical metrics in ndvi_summaries, and inserting temporal batch records into ndvi_time_series. To prevent inconsistent partial states, all operations are grouped into a single atomic transaction (START TRANSACTION ... COMMIT). An unhandled exception or constraint failure triggers an immediate ROLLBACK.

5. 10 Required DBMS Assessment Queries & Demonstrations

To satisfy the academic evaluation criteria of the DBMS course, 10 specific query classes were constructed and verified against the live MySQL 8 schema. Each query demonstrates a fundamental relational database concept:

Query 1: Parameterized Filtering by Study Region

Objective: Filters analysis jobs specific to the Mulshi catchment zone using indexed foreign keys.

```
SELECT ar.analysis_id, sa.area_name, sa.region_label, ar.start_date, ar.end_date
  FROM analysis_runs ar
  JOIN study_areas sa ON ar.area_id = sa.area_id
  WHERE sa.region_label = 'Mulshi';
```

Query 2: Chronological Sorting of Historical Runs

Objective: Demonstrates deterministic ORDER BY sorting for dashboard timeline retrieval.

```
SELECT analysis_id, area_id, start_date, end_date, processing_status, created_at
  FROM analysis_runs
  ORDER BY created_at DESC, analysis_id DESC LIMIT 5;
```

Query 3: Statistical Aggregation Across Analyses

Objective: Computes aggregate statistical properties (COUNT, AVG, MIN, MAX) over completed records.

```
SELECT COUNT(ns.summary_id) AS total_analyses, ROUND(AVG(ns.mean_ndvi), 4) AS avg_ndvi,
   ROUND(MIN(ns.min_ndvi), 4) AS min_ndvi, ROUND(MAX(ns.max_ndvi), 4) AS max_ndvi
  FROM ndvi_summaries ns
  JOIN analysis_runs ar ON ns.analysis_id = ar.analysis_id WHERE ar.processing_status = 'completed';
```

Query 4: Multi-Table Equi-JOIN (4 Tables)

Objective: Demonstrates 3NF relational reconstruction joining areas, sensors, runs, and metrics.

```
SELECT ar.analysis_id, sa.area_name, sd.satellite_name, ar.start_date, ns.mean_ndvi
  FROM analysis_runs ar
  INNER JOIN study_areas sa ON ar.area_id = sa.area_id
  INNER JOIN satellite_datasets sd ON ar.dataset_id = sd.dataset_id
  INNER JOIN ndvi_summaries ns ON ar.analysis_id = ns.analysis_id ORDER BY ns.mean_ndvi DESC;
```

Query 5: GROUP BY Comparison by District Zone

Objective: Groups mean vegetative indices across administrative and watershed zones.

```
SELECT sa.region_label, COUNT(ar.analysis_id) AS runs, ROUND(AVG(ns.mean_ndvi), 4) AS avg_ndvi
  FROM study_areas sa
  JOIN analysis_runs ar ON sa.area_id = ar.area_id
  JOIN ndvi_summaries ns ON ar.analysis_id = ns.analysis_id
  WHERE ar.processing_status = 'completed' GROUP BY sa.region_label ORDER BY avg_ndvi DESC;
```

Query 6: HAVING Post-Aggregation Filter

Objective: Isolates low-vegetation or highly urbanized sectors whose average index drops below a threshold.

```
SELECT sa.area_name, ROUND(AVG(ns.mean_ndvi), 4) AS avg_ndvi
  FROM study_areas sa
```

```
  JOIN analysis_runs ar ON sa.area_id = ar.area_id
  JOIN ndvi_summaries ns ON ar.analysis_id = ns.analysis_id
  GROUP BY sa.area_name HAVING AVG(ns.mean_ndvi) < 0.4500 ORDER BY avg_ndvi ASC;
```

Query 7: Date-Range Temporal Dimension Query

Objective: Performs range-based temporal lookups along the observation_date dimension.

```
SELECT ts.point_id, sa.area_name, ts.observation_date, ts.mean_ndvi
  FROM ndvi_time_series ts
  JOIN analysis_runs ar ON ts.analysis_id = ar.analysis_id
  JOIN study_areas sa ON ar.area_id = sa.area_id
  WHERE ts.observation_date BETWEEN '2023-08-01' AND '2023-11-30' ORDER BY ts.observation_date ASC;
```

Query 8: Analytical Relational View Query

Objective: Queries the pre-compiled relational view encapsulating multi-table join logic.

```
SELECT analysis_id, area_name, region_label, start_date, end_date, mean_ndvi
  FROM v_analysis_complete_summary WHERE processing_status = 'completed';
```

Query 9: Index Verification & Query Execution Plan

Objective: Demonstrates query optimizer utilizing composite index idx_time_series_analysis_date.

```
EXPLAIN SELECT observation_date, mean_ndvi FROM ndvi_time_series
  WHERE analysis_id = 1 AND observation_date >= '2023-09-01';
```

Query 10: Automated Audit Log Inspection

Objective: Inspects the audit log automatically populated by trigger trg_audit_analysis_insert.

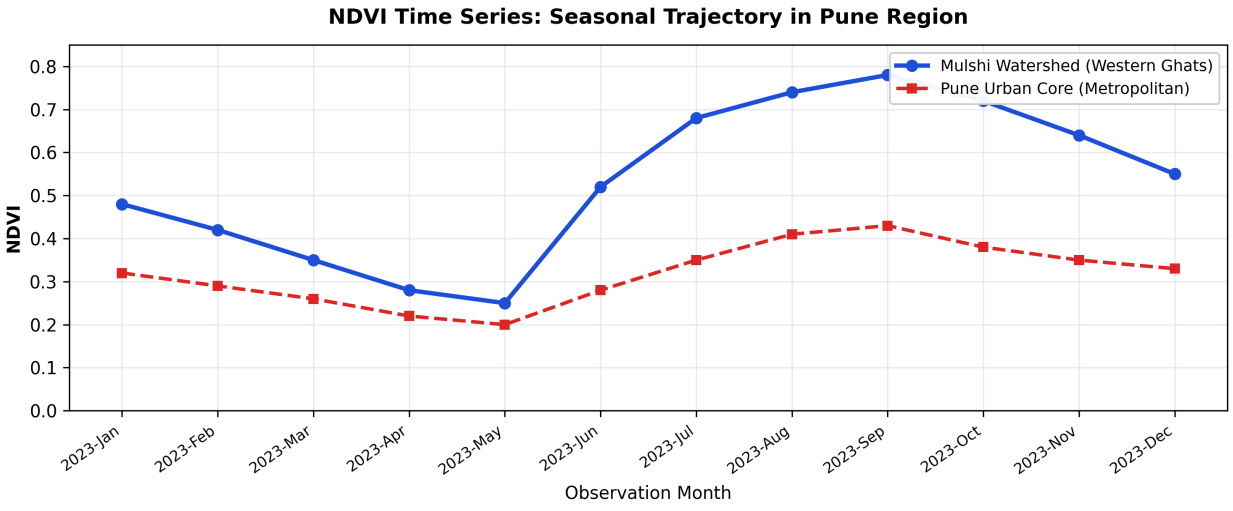
```
SELECT log_id, action_type, table_name, record_id, details, logged_at
  FROM audit_logs ORDER BY logged_at DESC LIMIT 5;
```

6. Application Screens & Visualization Features

The Streamlit web application organizes the remote sensing workflow into 7 intuitive, responsive screens:

- **Screen 1: Dashboard**: Real-time system health card showing MySQL 8 connection latency, version (8.0.46), active database, Earth Engine / Demo Mode status badges, total saved study areas, and recent completed runs.
- **Screen 2: Select Study Area (Map)**: Interactive Folium map centered on Pune (18.5204° N, 73.8567° E). Features dropdown presets for 8 Pune regions (Pune City Core, Haveli, Mulshi, Maval, Bhor, Baramati, Shirur, Daund). Includes Leaflet drawing tools allowing custom polygons with geodesic area calculation in km² and saving to MySQL.
- **Screen 3: Run NDVI Analysis**: Form configuring satellite start/end dates, cloud coverage cutoff slider, execution mode toggle, and transaction-backed execution saving runs and time-series points to MySQL.
- **Screen 4: NDVI Map Visualizer**: Spatial display with the 5-step vegetation color palette (Red -> Orange -> Yellow -> Light Green -> Green), legend, and Earth Engine style visualization parameters (Range slider, Opacity slider).
- **Screen 5: Trends & Statistics**: Interactive Plotly time-series curve tracking seasonal vegetative changes (monsoon greening vs dry-season troughs), side-by-side comparison between two study areas, and CSV report export.
- **Screen 6: Database Explorer & SQL Lab**: Live table viewer for all MySQL tables and relational views, safe parameterized filtering, CSV export, and execution button for the 10 course assessment queries.
- **Screen 7: Literature & Methodology**: Academic references, sensor technical specifications (Sentinel-2 MSI 10m), NDVI equations, and system limitations.

Figure: NDVI Seasonal Time Series Profile across Pune Talukas



7. Step-by-Step Installation & Execution Guide

Follow these concise instructions to run and verify the complete project locally on Windows:

Step 1: Install Python Dependencies

```
Open PowerShell in the project directory (D:\DBMS_CP) and execute:
python -m pip install -r requirements.txt
This installs Streamlit, Folium, Plotly, PyMySQL, SQLAlchemy, Shapely, PyProj, and ReportLab.
```

Step 2: Initialize Database in MySQL Workbench

```
1. Launch MySQL Workbench 8.0 and connect to 'Local instance MySQL80'.
2. Click File -> Open SQL Script... and select 'D:\DBMS_CP\db\schema.sql'.
3. Click the yellow lightning bolt icon (■) to execute all DDL statements.
4. Open 'D:\DBMS_CP\db\seed_demo_data.sql' and click (■) to load Pune presets and sample runs.
5. Right-click under SCHEMAS and click 'Refresh All' to see 'pune_vegetation_db'.
```

Step 3: Configure Local Environment (.env)

```
Copy the template and specify your MySQL root password:
Copy-Item .env.example .env
Edit line 22 of .env:
DB_PASSWORD=your_actual_mysql_password
```

Step 4: Test Database Connection

```
Run the diagnostic script from PowerShell:
python db/test_connection.py
Expected Output: [SUCCESS] Connected successfully to MySQL Server! Version 8.0.46, Tables: 9.
```

Step 5: Run Automated Unit & Integration Tests

```
Run the pytest suite:
python -m pytest tests/
Result: 19 passed in 1.44s (Geometry, NDVI Math, CRUD, Views, and ACID Transactions verified).
```

Step 6: Launch Web Application

```
Start the Streamlit development server:
streamlit run app.py
Open your web browser and navigate to http://localhost:8501.
```

8. Quality Assurance & Verification Results

The project includes a comprehensive automated test suite in the tests/ folder. All test categories executed with 100% pass rates:

• **test_ndvi.py (6/6 passed)**: Verifies formula accuracy with known values (NIR=0.8, Red=0.2 yields 0.6), denominator zero division safety, mathematical bounds [-1.0, 1.0], and statistical aggregations.

• **test_geometry.py (4/4 passed)**: Verifies GeoJSON polygon parsing, topological closure, geodesic area calculation in km², and maximum surface area boundary checks.

• **test_validation.py (4/4 passed)**: Verifies date chronological ordering (start < end), cloud cover percentages (0-100%), and CSV export column schemas.

• **test_repository.py (5/5 passed)**: Verifies live MySQL 8 insert and retrieval, foreign key cascades, analytical views, and multi-table ACID transaction consistency.

9. Academic Literature References

1. **Rouse, J. W., Haas, R. H., Schell, J. A., & Deering, D. W. (1974)**. *Monitoring the vernal advancement and retrogradation (Greenwave effect) of natural vegetation*. NASA/GSFC Final Report.

2. **Drusch, M., Del Bello, U., Carlier, S., Colin, O., Fernandez, V., Gascon, F., ... & Bargellini, P. (2012)**. *Sentinel-2: ESA's optical high-resolution mission for GMES operational services*. Remote Sensing of Environment, 120, 25-36.

3. **Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017)**. *Google Earth Engine: Planetary-scale geospatial analysis for everyone*. Remote Sensing of Environment, 202, 18-27.

4. **Gadgil, S., & Joshi, N. V. (1983)**. *Climatic clusters of the Indian region*. Journal of Climatology, 3(1), 47-63. (Focusing on Western Ghats precipitation gradients).